

SCRIPT DE SOUTENANCE

Projet 1 — Pipeline de Données Industrielles

Safran Data Systems · Dataset NASA CMAPSS · Compétences C1 à C5 · Durée : 10 minutes

Minutage global

Slide	Sujet	Durée	Compétence
1	Titre & introduction	30 sec	—
2	Contexte & architecture	1 min	—
3	Collecte multi-sources	2 min	C1
4	Requêtes SQL	1 min	C2
5	Nettoyage & agrégation	1 min 30	C3
6	Base de données	1 min	C4
7	API REST	1 min 30	C5
8	Démo API	30 sec	C5
9	Pipeline industrialisé	30 sec	C5
10-12	Récap + Conclusion	30 sec	—
TOTAL		~10 min	

■ Conseil : Répétez à voix haute 2 fois avant la soutenance. Chronométrez-vous sur les slides C1 et C5 qui sont les plus denses.

SLIDE 1 — TITRE

■ 30 secondes

Bonjour,

Je vais vous présenter mon premier projet : un pipeline de données industrielles réalisé dans le contexte de Safran Data Systems.

L'objectif de ce projet était de concevoir et développer un système complet permettant de collecter, nettoyer, stocker et exposer des données capteurs avioniques issues de sources hétérogènes.

Ce projet me permet de valider les compétences C1 à C5 du bloc 1 du référentiel Développeur en Intelligence Artificielle.

→ *Regarder le jury, parler lentement, sourire.*

SLIDE 2 — CONTEXTE & ARCHITECTURE

■ 1 minute

Le contexte.

Safran Data Systems équipe des avions avec des capteurs qui génèrent en continu des données de température, vibration et pression. Le problème : ces données sont dispersées, non harmonisées, difficiles à exploiter.

Pour simuler ce contexte industriel, j'ai utilisé le dataset NASA CMAPSS, qui contient des mesures de dégradation de moteurs turbofan — un dataset réel utilisé dans l'industrie aéronautique.

L'architecture du pipeline se décompose en 4 phases :

- Phase 1 : collecte multi-sources
- Phase 2 : nettoyage et normalisation
- Phase 3 : stockage en base de données
- Phase 4 : exposition via une API REST

En chiffres : 123 886 lignes traitées, 5 sources de données, 4 endpoints API, et 5 compétences validées.

La stack technique : Python 3.13, pandas, FastAPI, SQLite, BeautifulSoup4.

→ *Pointer les 4 phases sur le slide en les énumérant.*

SLIDE 3 — C1 — COLLECTE MULTI-SOURCES

■ 2 minutes

Compétence C1 — Automatiser l'extraction de données depuis plusieurs sources.

Le référentiel exige une collecte depuis un mix d'au moins 5 types de sources. J'ai implémenté exactement ça dans le script `collect_data.py`.

Source 1 — API REST.

J'interroge l'API publique JSONPlaceholder pour simuler une API interne Safran. Le script construit la requête HTTP, parse le JSON et retourne un DataFrame de 100 lignes.

Source 2 — Fichiers CSV NASA.

Les fichiers CMAPSS sont chargés automatiquement depuis data/raw/. Le script les détecte tous via un glob et les concatène. Résultat : 61 893 lignes.

Source 3 — Base SQL SQLite.

J'exécute programmatiquement une requête SQL sur la base source.db via Python. 20 631 lignes extraites.

Source 4 — Big Data simulé.

J'ai dupliqué les fichiers NASA dans data/raw/bigdata/ pour simuler un volume big data. 41 262 lignes supplémentaires.

Source 5 — Scraping web.

C'est la source qui prouve la maîtrise du scraping. J'utilise BeautifulSoup4 pour extraire l'infobox Wikipedia du moteur CFM56 — un moteur co-développé par Safran et GE Aviation. Le script gère le User-Agent, parse le HTML et extrait les données du tableau.

Comme vous le voyez sur la capture, les 5 sources s'exécutent en séquence et les fichiers CSV intermédiaires sont sauvegardés dans data/processed/.

→ Pointer la capture terminal sur le slide. Insister sur le scraping : c'est le critère le plus visible.

SLIDE 4 — C2 — REQUÊTES SQL

■ 1 minute

Compétence C2 — Développer des requêtes SQL d'extraction documentées.

J'ai mis en place deux niveaux de requêtes SQL.

Côté collecte :

La requête sur source.db est un SELECT exhaustif sans filtre. Ce choix est justifié : on collecte brut à ce stade, le nettoyage étant délégué à C3. Il n'y a pas de jointure car la table capteurs est l'unique source SQL.

Côté API :

J'applique deux optimisations importantes. D'abord, la pagination avec LIMIT pour éviter de charger tout en mémoire. Ensuite, et c'est crucial en termes de sécurité, j'utilise des requêtes paramétrées avec le point d'interrogation comme placeholder. Ça protège contre les injections SQL — au lieu d'interpoler directement unit_id dans la chaîne, la valeur est liée séparément par SQLite. C'est une bonne pratique OWASP que j'ai intégrée.

→ Pointer le WHERE unit = ? dans le code sur le slide. Mentionner OWASP spontanément, ça montre la culture sécurité.

SLIDE 5 — C3 — NETTOYAGE & AGRÉGATION

■ 1 minute 30

Compétence C3 — Développer des règles d'agrégation et de nettoyage.

Le nettoyage est réalisé dans `aggregate_data.py` via la fonction `clean_dataframe()`. J'applique 4 opérations sur chaque source avant fusion.

Opération 1 — Suppression des colonnes vides.

Les fichiers NASA CMAPSS contiennent 2 colonnes entièrement vides dues au format d'export. `dropna(axis=1, how='all')` les élimine automatiquement.

Opération 2 — Suppression des lignes vides.

Pour éviter les enregistrements fantômes, `dropna(how='all')` supprime toute ligne entièrement nulle.

Opération 3 — Gestion des valeurs manquantes.

Pour les colonnes numériques, j'impute par la moyenne. C'est une méthode classique qui préserve la distribution statistique sans distordre les capteurs.

Opération 4 — Harmonisation des types.

Les colonnes de type object sont converties en string pour éviter les erreurs lors de la fusion multi-sources hétérogènes.

Une fois chaque source nettoyée, `pd.concat()` fusionne les 5 DataFrames en un seul. Résultat visible sur la capture : 123 886 lignes propres, exportées dans `final_clean_data.csv`.

→ Pointer le code `clean_dataframe()` et la ligne de résultat terminal. Insister sur le chiffre 123 886.

SLIDE 6 — C4 — BASE DE DONNÉES

■ 1 minute

Compétence C4 — Créer une base de données avec modélisation Merise.

Avant de créer la base, j'ai produit un MCD et un MPD selon la méthode Merise.

Le MCD

identifie 4 entités : SOURCE, MESURE, UNITE_MOTEUR et PIPELINE_EXEC, avec leurs associations et cardinalités.

Le MPD

traduit ça en SQL : clés primaires, clés étrangères, types de colonnes INTEGER et REAL, et les relations 1,N entre tables.

Le choix de SQLite

est justifié par le contexte : déploiement local sans serveur, volume de 123 000 lignes parfaitement adapté, et intégration native avec Python et FastAPI.

Le script `import_to_db.py` charge le CSV nettoyé et l'insère dans la table mesures de `final.db`. Comme vous le voyez sur la capture : base créée, table importée avec succès.

Concernant le RGPD :

les données utilisées sont exclusivement des données techniques de capteurs NASA, des données publiques Wikipedia et des données fictives. Aucune donnée personnelle n'est traitée. Le registre des traitements est donc vide, ce que j'ai documenté explicitement dans les spécifications techniques.

→ Pointer MCD et MPD. Mentionner le RGPD spontanément — le jury le note.

SLIDE 7 — C5 — API REST

■ 1 minute 30

Compétence C5 — Développer une API REST mettant à disposition les données.

L'API est développée avec FastAPI et expose 4 endpoints documentés via Swagger UI au standard OpenAPI 3.1.

L'authentification

est implémentée via le mécanisme APIKeyHeader. Chaque requête sur les routes protégées doit inclure le header X-API-Key. Sans clé valide, l'API retourne un HTTP 401. Vous voyez sur la capture le cadenas présent sur chaque endpoint sécurisé, et la fenêtre d'autorisation Swagger.

Les 4 endpoints :

- GET / → status de l'API, public
- GET /mesures → échantillon paginé, protégé
- GET /mesures/{unit_id} → filtrage par unité moteur, protégé, retourne 404 si introuvable
- GET /stats → statistiques globales, protégé

La clé API est stockée dans le fichier .env et chargée via python-dotenv — bonne pratique de sécurité pour ne pas exposer de credentials dans le code versionné sur Git.

→ Pointer le cadenas Swagger sur le slide. Montrer le bouton Authorize. Mentionner .env et Git spontanément.

SLIDE 8 — C5 — DÉMO API

■ 30 secondes

Sur cette slide vous voyez les réponses réelles de l'API en temps réel.

À gauche — GET /mesures :

retourne un JSON avec les champs capteurs NASA, les paramètres opérationnels, et les colonnes de provenance. Code 200, header X-API-Key visible dans le curl généré par Swagger.

À droite — GET /stats :

retourne en temps réel : 123 886 mesures totales, 100 unités moteur distinctes. Simple, efficace, directement exploitable par les équipes data Safran.

→ Slide rapide, ne pas s'attarder. Pointer les deux réponses JSON.

SLIDE 9 — PIPELINE INDUSTRIALISÉ

■ 30 secondes

L'industrialisation, c'est ce qui distingue un script d'un vrai pipeline.

run_pipeline.py orchestre les 3 étapes en une seule commande. Chaque étape est encapsulée dans un try/except : si la collecte échoue, l'agrégation tente quand même de s'exécuter avec les données disponibles.

Le système de logs horodatés dans pipeline.log trace chaque exécution avec timestamp, étape et statut. Sur la capture, vous voyez l'historique complet d'une exécution : démarrage, collecte, agrégation, import SQL, et confirmation de succès.

→ Slide rapide. Pointer le terminal run_pipeline et les logs horodatés.

SLIDE 10 — RÉCAP COMPÉTENCES C1 → C5

■ 30 secondes

Pour résumer, ce projet valide les 5 compétences du bloc 1 :

- C1 : 5 sources collectées incluant le scraping obligatoire
- C2 : requêtes SQL documentées et sécurisées
- C3 : 4 opérations de nettoyage, 123 886 lignes normalisées
- C4 : modélisation Merise complète, base SQLite opérationnelle
- C5 : API REST avec authentification et documentation OpenAPI 3.1

→ Pointer chaque colonne du tableau récap sur le slide.

SLIDE 11-12 — SUITE & CONCLUSION

■ 30 secondes

Ce projet constitue la base technique sur laquelle s'appuieront les 3 projets suivants.

- Le projet 2 intégrera un service IA préexistant en s'appuyant sur cette API.
- Le projet 3 déploiera un modèle avec une approche MLOps.
- Le projet 4 développera une application complète de bout en bout.

En conclusion :

Ce pipeline prouve la maîtrise de l'ensemble de la chaîne data engineer — de la collecte brute jusqu'à l'exposition sécurisée des données, en passant par la modélisation, le nettoyage et l'industrialisation.

Merci de votre attention. Je suis disponible pour vos questions.

→ Regarder le jury. Sourire. Poser les mains sur la table. Attendre les questions calmement.

Questions fréquentes du jury — Prépare tes réponses

Q : Pourquoi SQLite et pas PostgreSQL ou MongoDB ?

R : Le volume de données (~123 000 lignes) ne justifie pas un serveur. SQLite est suffisant, sans dépendance externe, et s'intègre nativement avec Python et FastAPI. En production sur un vrai système Safran, on migrerait vers PostgreSQL.

Q : Pourquoi imputer par la moyenne et pas par la médiane ?

R : La moyenne est adaptée aux données de capteurs physiques qui suivent des distributions relativement symétriques. La médiane serait plus robuste si on avait des outliers importants, mais sur des données industrielles calibrées, la moyenne est un choix standard et justifiable.

Q : Ton scraping Wikipedia, c'est vraiment du big data ?

R : Non, le scraping n'est pas du big data. Ce sont deux sources distinctes. Le big data est simulé par la duplication des fichiers NASA dans `data/raw/bigdata/`. Le scraping Wikipedia est la 5ème source qui répond à l'exigence du référentiel d'inclure une source web scrapée.

Q : Comment tu gères la sécurité de ta clé API ?

R : La clé est stockée dans un fichier `.env` non versionné sur Git. Elle est chargée via `python-dotenv` au démarrage de l'API. En production, on utiliserait un gestionnaire de secrets comme Vault ou les secrets Kubernetes. Le `.gitignore` exclut le `.env`.

Q : Quid du RGPD sur ce projet ?

R : Aucune donnée personnelle n'est traitée. Le dataset NASA CMAPSS est entièrement anonymisé — ce sont des mesures techniques de capteurs. Les données JSONPlaceholder sont fictives. Wikipedia est public. Le registre des traitements est donc vide, ce que j'ai documenté dans les spécifications.

Q : Comment tu améliorerais ce pipeline en production ?

R : Plusieurs axes : remplacer SQLite par PostgreSQL, containeriser avec Docker, ajouter des tests unitaires `pytest`, mettre en place une vraie CI/CD GitHub Actions, monitorer avec Prometheus/Grafana, et remplacer le scraping Wikipedia par une vraie API industrielle Safran.